Contents

1 Ant Stack Implementation Appendix **1**
- 1.1 Introduction . . . 1
- 1.2 Layer-by-Layer Integration . . . 2
  - 1.2.1 AntBody Layer: Physical Simulation and Sensing . . . 2
  - 1.2.2 AntBrain Layer: Neural Architecture . . . 3
  - 1.2.3 AntMind Layer: Cognitive Modeling . . . 4
- 1.3 Species-Specific Implementations . . . 5
  - 1.3.1 Formica Species Configuration . . . 5
  - 1.3.2 Camponotus Species Configuration . . . 6
- 1.4 Evaluation and Benchmarking . . . 7
  - 1.4.1 Navigation Performance Metrics . . . 7
  - 1.4.2 Robustness Testing . . . 8
- 1.5 Implementation Workflow . . . 8
  - 1.5.1 Development Pipeline . . . 8
  - 1.5.2 Code Organization . . . 9
- 1.6 Integration Benefits . . . 10
  - 1.6.1 Reproducibility . . . 10
  - 1.6.2 Extensibility . . . 10
  - 1.6.3 Validation . . . 10
- 1.7 Future Directions . . . 10
  - 1.7.1 Advanced Learning Mechanisms . . . 10
  - 1.7.2 Hardware Integration . . . 10
  - 1.7.3 Biological Validation . . . 10
- 1.8 Conclusion . . . 10

1 Ant Stack Implementation Appendix

1.1 Introduction

This appendix provides a comprehensive mapping of how the cohereAnts research framework can be implemented within "The Ant Stack" computational architecture. The Ant Stack, as described by Friedman (2025), offers a modular, three-layer framework for emulating ant intelligence from physical embodiment to collective cognition. Here we demonstrate how our vibrational theory of olfaction, spectroscopic analysis, and behavioral modeling can be systematically integrated into this standardized computational framework.

Implementation Strategy: We map cohereAnts modules to Ant Stack layers using explicit I/O contracts, ensuring reproducible experiments while maintaining the biological plausibility of our theoretical framework.

1.2 Layer-by-Layer Integration

1.2.1 AntBody Layer: Physical Simulation and Sensing

Sensilla Morphology Integration The cohereAnts sensilla analysis module maps directly to AntBody's morphological modeling:

```
# AntBody sensilla configuration
class AntBodySensilla:
    def __init__(self, species_preset: str):
        # Load species-specific sensilla parameters
        self.lengths = load_sensilla_lengths(species_preset)
        self.diameters = load_sensilla_diameters(species_preset)
        self.optimal_wavelengths = self._calculate_resonance()

    def _calculate_resonance(self):
        # Implement cohereAnts resonance theory
        return self.lengths * 4  # Quarter-wavelength resonance
```

I/O Contract: - **Observations**: Sensilla dimensions (m), resonance frequencies (THz), quality factors - **Actions**: Antenna positioning, sensilla orientation - **Physics**: 1 kHz update rate, contact dynamics for substrate interaction

Spectroscopy and Atmospheric Transmission Integration of cohereAnts atmospheric transmission models:

```
class AntBodySpectroscopy:
    def __init__(self, environment_preset: str):
        self.transmission_curves = load_atmospheric_data(environment_preset)
        self.spectral_resolution = 0.01  #  m

    def get_transmission(self, wavelength: float, distance: float) -> float:
        # Implement cohereAnts atmospheric transmission model
        return calculate_atmospheric_transmission(wavelength, distance)
```

Configuration Parameters: - Atmospheric windows: 2-5 m, 8-14 m, 17-25 m - Transmission coefficients: 0.7-0.9 for optimal windows - Distance-dependent attenuation models

1.2.2 AntBrain Layer: Neural Architecture

Olfactory Processing Pipeline Mapping cohereAnts vibrational theory to AntBrain's AL→MB→CX architecture:

```
class AntBrainOlfaction:
    def __init__(self, neuron_count: int = 100000):
        # Antennal Lobe (AL) - odor coding
        self.al_neurons = self._initialize_al_circuit()
        # Mushroom Body (MB) - associative learning
        self.mb_neurons = self._initialize_mb_circuit()
        # Central Complex (CX) - spatial integration
        self.cx_neurons = self._initialize_cx_circuit()

    def _initialize_al_circuit(self):
        # Implement cohereAnts vibrational detection theory
        # Each glomerulus responds to specific molecular vibrations
        return VibrationalGlomeruliCircuit()

    def _initialize_mb_circuit(self):
        # Kenyon cells for odor-memory associations
        return KenyonCellCircuit()

    def _initialize_cx_circuit(self):
        # Ring attractor for heading representation
        return RingAttractorCircuit()
```

Neural Implementation Details: - **AL Layer**: 50 glomeruli, each tuned to specific vibrational frequencies - **MB Layer**: 2500 Kenyon cells with sparse coding (5% activity) - **CX Layer**: 16-heading ring attractor with 100 neurons per heading

Vibrational Detection Circuit Implementation of cohereAnts electromagnetic theory:

```
class VibrationalGlomeruliCircuit:
    def __init__(self):
        self.frequency_tuning = np.linspace(2, 25, 50)  #  m to THz
        self.quality_factors = np.ones(50) * 100

    def process_spectral_input(self, spectral_data: np.ndarray) -> np.ndarray:
```

```python
        # Implement cohereAnts resonance detection
        responses = np.zeros(50)
        for i, freq in enumerate(self.frequency_tuning):
            responses[i] = self._calculate_vibrational_response(spectral_data, freq)
        return responses

    def _calculate_vibrational_response(self, spectrum: np.ndarray,
                                        resonant_freq: float) -> float:
        # Apply cohereAnts electromagnetic coupling model
        coupling_strength = self._calculate_coupling(spectrum, resonant_freq)
        return coupling_strength * self.quality_factors[i]
```

1.2.3 AntMind Layer: Cognitive Modeling

Active Inference for Olfactory Search Integration of cohereAnts behavioral models with active inference:

```python
class AntMindOlfaction:
    def __init__(self):
        self.generative_model = self._build_olfactory_model()
        self.policy_horizon = 2.0  # seconds

    def _build_olfactory_model(self):
        # Implement cohereAnts behavioral predictions
        return OlfactoryGenerativeModel()

    def select_policy(self, current_state: Dict) -> np.ndarray:
        # Active inference policy selection
        expected_free_energy = self._calculate_efe()
        return self._minimize_free_energy(expected_free_energy)

    def _calculate_efe(self) -> Dict[str, float]:
        # Decompose into epistemic and pragmatic value
        return {
            'epistemic': self._calculate_epistemic_value(),
            'pragmatic': self._calculate_pragmatic_value()
        }
```

Stigmergy for Trail Following Implementation of cohereAnts pheromone dynamics:

```python
class AntMindStigmergy:
    def __init__(self):
        self.pheromone_field = np.zeros((100, 100))
        self.decay_rate = 0.01
        self.diffusion_coefficient = 0.1

    def update_pheromone_field(self, deposits: List[Tuple[int, int, float]]):
        # Implement cohereAnts pheromone diffusion model
        for x, y, amount in deposits:
            self.pheromone_field[x, y] += amount

        # Apply diffusion and decay
        self.pheromone_field = self._diffuse_and_decay()

    def _diffuse_and_decay(self) -> np.ndarray:
        # Fick's law implementation from cohereAnts
        laplacian = self._calculate_laplacian(self.pheromone_field)
        diffusion = self.diffusion_coefficient * laplacian
        decay = -self.decay_rate * self.pheromone_field
        return self.pheromone_field + diffusion + decay
```

1.3 Species-Specific Implementations

1.3.1 Formica Species Configuration

```python
# Formica species preset for Ant Stack
FORMICA_PRESET = {
    'body': {
        'sensilla_lengths': [15.2, 18.7, 22.1, 19.8, 16.5],  # m
        'sensilla_diameters': [2.1, 2.8, 3.2, 2.9, 2.3],     # m
        'optimal_wavelengths': [60.8, 74.8, 88.4, 79.2, 66.0],  # m
        'antenna_length': 2.5,  # mm
        'leg_count': 6,
        'body_mass': 0.015  # g
```

```
    },
    'brain': {
        'al_glomeruli_count': 50,
        'mb_kenyon_cells': 2500,
        'cx_heading_resolution': 16,
        'spiking_threshold': 0.1,
        'learning_rate': 0.01
    },
    'mind': {
        'policy_horizon': 2.0,  # seconds
        'pheromone_decay': 0.01,
        'diffusion_coefficient': 0.1,
        'exploration_rate': 0.2
    }
}
```

1.3.2 Camponotus Species Configuration

```
# Camponotus species preset for Ant Stack
CAMPONOTUS_PRESET = {
    'body': {
        'sensilla_lengths': [22.5, 28.1, 31.7, 26.8, 24.3],  # m
        'sensilla_diameters': [3.2, 4.1, 4.8, 4.2, 3.6],      # m
        'optimal_wavelengths': [90.0, 112.4, 126.8, 107.2, 97.2], #  m
        'antenna_length': 3.8,  # mm
        'leg_count': 6,
        'body_mass': 0.045  # g
    },
    'brain': {
        'al_glomeruli_count': 60,
        'mb_kenyon_cells': 3000,
        'cx_heading_resolution': 20,
        'spiking_threshold': 0.08,
        'learning_rate': 0.015
    },
    'mind': {
        'policy_horizon': 2.5,  # seconds
```

```
        'pheromone_decay': 0.008,
        'diffusion_coefficient': 0.12,
        'exploration_rate': 0.15
    }
}
```

1.4 Evaluation and Benchmarking

1.4.1 Navigation Performance Metrics

```
class AntStackEvaluator:
    def __init__(self, test_scenarios: List[str]):
        self.scenarios = test_scenarios
        self.metrics = {}

    def evaluate_navigation(self, ant_stack: AntStack) -> Dict[str, float]:
        results = {}
        for scenario in self.scenarios:
            if scenario == 'trail_following':
                results[scenario] = self._evaluate_trail_following(ant_stack)
            elif scenario == 'food_search':
                results[scenario] = self._evaluate_food_search(ant_stack)
            elif scenario == 'nest_return':
                results[scenario] = self._evaluate_nest_return(ant_stack)
        return results

    def _evaluate_trail_following(self, ant_stack: AntStack) -> float:
        # Implement cohereAnts trail following metrics
        trail_deviation = self._calculate_trail_deviation()
        pheromone_detection = self._calculate_pheromone_detection()
        return self._combine_metrics([trail_deviation, pheromone_detection])

    def _evaluate_food_search(self, ant_stack: AntStack) -> float:
```

```
        # Implement cohereAnts search efficiency metrics
        search_time = self._measure_search_time()
        energy_efficiency = self._calculate_energy_efficiency()
        return self._combine_metrics([search_time,
           energy_efficiency])
```

1.4.2 Robustness Testing

```
class RobustnessTester:
    def __init__(self):
        self.noise_levels = [0.01, 0.05, 0.1, 0.2]
        self.adversary_types = ['sensor_noise', '
           pheromone_contamination', 'path_obstruction']

    def test_noise_robustness(self, ant_stack: AntStack) ->
       Dict[str, float]:
        results = {}
        for noise_level in self.noise_levels:
            performance = self._run_noisy_scenario(ant_stack,
               noise_level)
            results[f'noise_{noise_level}'] = performance
        return results

    def test_adversary_robustness(self, ant_stack: AntStack) ->
        Dict[str, float]:
        results = {}
        for adversary in self.adversary_types:
            performance = self._run_adversarial_scenario(
               ant_stack, adversary)
            results[f'adversary_{adversary}'] = performance
        return results
```

1.5 Implementation Workflow

1.5.1 Development Pipeline

1. **Module Mapping**: Identify cohereAnts functions for Ant Stack integration
2. **I/O Contract Definition**: Establish standardized interfaces between layers
3. **Species Preset Creation**: Develop parameterized configurations
4. **Testing Framework**: Implement evaluation metrics and benchmarks
5. **Documentation**: Create implementation guides and examples

1.5.2 Code Organization

```
ant_stack_cohereants/
          antbody/
                sensilla_physics.py      # cohereAnts vibrational theory
                spectroscopy_sensors.py  # atmospheric transmission models
                morphology_models.py     # species-specific parameters
          antbrain/
                olfactory_circuits.py    #   A  L  MBCX implementation
                vibrational_detection.py # electromagnetic coupling
                learning_mechanisms.py   # STDP and plasticity
          antmind/
                olfactory_inference.py   # active inference models
                stigmergy_models.py      # pheromone dynamics
                behavioral_policies.py   # search and navigation
          presets/
                formica_config.py        # Formica species preset
                camponotus_config.py     # Camponotus species preset
                custom_species.py        # Template for new species
          evaluation/
              navigation_tests.py      # Trail following, search
              robustness_tests.py      # Noise, adversary testing
              performance_metrics.py   # Standardized benchmarks
```

1.6 Integration Benefits

1.6.1 Reproducibility

- **Standardized I/O**: All experiments use consistent interfaces
- **Version Pinning**: Dependencies and parameters are explicitly tracked
- **Seed Management**: Reproducible random number generation
- **Artifact Tracking**: Complete experiment provenance

1.6.2 Extensibility

- **Species Presets**: Easy addition of new ant species
- **Module Swapping**: Interchangeable components across layers
- **Parameter Tuning**: Systematic exploration of parameter space
- **Benchmark Addition**: New evaluation scenarios

1.6.3 Validation

- **Biological Plausibility**: Grounded in empirical data
- **Performance Metrics**: Quantified success criteria
- **Robustness Testing**: Resilience to real-world challenges
- **Cross-Species Transfer**: Generalization across taxa

1.7 Future Directions

1.7.1 Advanced Learning Mechanisms

- **Meta-Learning**: Adaptation across different environments
- **Collective Intelligence**: Emergent behaviors in colonies
- **Transfer Learning**: Knowledge transfer between species

1.7.2 Hardware Integration

- **Robotic Platforms**: Physical ant-inspired robots
- **Sensor Networks**: Distributed environmental monitoring
- **Edge Computing**: Efficient on-device processing

1.7.3 Biological Validation

- **Field Studies**: Comparison with natural ant behavior
- **Neural Recording**: Validation against biological data
- **Evolutionary Analysis**: Phylogenetic patterns in behavior

1.8 Conclusion

The integration of cohereAnts research into the Ant Stack framework provides a robust, reproducible platform for studying ant intelligence. By mapping our vibrational theory of olfaction, spectroscopic analysis, and behavioral modeling to the standardized three-layer architecture, we create a comprehensive system that bridges theoretical insights with computational implementation.

This implementation enables systematic exploration of ant behavior across species, environments, and experimental conditions while maintaining the biological plausibility that underpins our research. The modular design facilitates both hypothesis testing in myrmecology and applications in swarm robotics, cognitive security, and AI alignment.

Key Contributions: 1. **Systematic Integration**: Methodical mapping of cohereAnts to Ant Stack layers 2. **Species Parameterization**: Reproducible configurations for multiple ant taxa 3. **Evaluation Framework**: Standardized metrics and robustness testing 4. **Implementation Workflow**: Clear development pipeline and code organization 5. **Future Roadmap**: Extensibility and validation pathways

The resulting framework serves as a bridge between theoretical entomology and computational neuroscience, enabling reproducible research that advances our understanding of both natural ant intelligence and artificial intelligence systems.